The **Model-Based Approach** stage using **Hugging Face Transformers** (specifically for **Named Entity Recognition** like `NAME`, `DATE_ISSUED`, `EXPIRY_DATE`), and considering you're working with documents that likely include structured layouts (like visas), here's how you can proceed—especially with **LayoutLMv3** for PDFs and standard token classification models for plain text.

✅ Step-by-Step Guide Based on Your Setup

🔁 1. Choose the Right Model Based on Input Type

A. For PDFs or Document Images (Scanned) → Use `LayoutLMv3`

- **Recommended Model**: `microsoft/layoutlmv3-base`
- **Use-case**: When layout information (e.g., positions of text blocks) is essential, like passports, visas, or forms.

B. For Plain Text → Use `bert-base-cased`, `roberta-base`, or other NER token classifiers

- **Recommended Model**: `dslim/bert-base-NER`, `Davlan/bert-base-multilingual-cased-ner-hrl`

📄 2. Preprocessing (depending on model type)

🅰 If PDF Input → Use `LayoutLMv3`

Install dependencies:

```
pip install transformers datasets torch torchvision
pip install pytesseract pdf2image
```

Convert PDF to image:

```
from pdf2image import convert_from_path
pages = convert_from_path("visa.pdf")
pages[0].save("page_0.png", "PNG")
```

Extract layout-aware input:

```
from transformers import LayoutLMv3Processor
from PIL import Image

image = Image.open("page_0.png")
processor = LayoutLMv3Processor.from_pretrained("microsoft/layoutlmv3-base",
apply_ocr=True)
encoding = processor(image, return_tensors="pt")
```

Run inference:

```
from transformers import LayoutLMv3ForTokenClassification

model = LayoutLMv3ForTokenClassification.from_pretrained("microsoft/layoutlmv3-
base", num_labels=NUM_CLASSES)
outputs = model(**encoding)
```

🅱 If Plain Text Input → Use `bert-base-NER`

Install:

```
pip install transformers
```

Code:

```
from transformers import pipeline

ner_pipe = pipeline("token-classification", model="dslim/bert-base-NER",
aggregation_strategy="simple")

text = """
Full Name : Mr. MOHAMED KHALAF ISMAIEL MOHAMED
Date & Place of Issue : 01 JAN 2022, CAIRO
Valid Until : 01 JAN 2024
"""

entities = ner_pipe(text)

for entity in entities:
    print(f"{entity['entity_group']}: {entity['word']} ({entity['score']:.2f})")
```

📤 3. Postprocessing: Extract `NAME`, `DATE_ISSUED`, `EXPIRY_DATE`

You can map predictions manually:

```
info = {"NAME": [], "DATE_ISSUED": None, "EXPIRY_DATE": None}

for ent in entities:
    if ent["entity_group"] == "PER":
        info["NAME"].append(ent["word"])
    elif ent["entity_group"] == "DATE":
        if info["DATE_ISSUED"] is None:
            info["DATE_ISSUED"] = ent["word"]
        else:
```

```
            info["EXPIRY_DATE"] = ent["word"]

info["NAME"] = " ".join(info["NAME"])
print(info)
```

📌 Tip for LayoutLMv3 Labels

If you're fine-tuning `LayoutLMv3`, your labels should be BIO-tagged (`B-NAME`, `I-NAME`, etc.). You'll need token-level annotations aligned with bounding boxes.

Would you like help with:

- 🛠 Fine-tuning LayoutLMv3 on your visa documents?
- 🧪 Running inference and getting results using Hugging Face NER models?